European Developer Conference



Presented By:
Ryan Robertson, Developer Technical Support

Joyce Morrell, Developer Programs





a new mobile entertainment platform to showcase your talents

palmsource

overview

company introduction
target customer
hardware overview
software environment
development tools API overview
porting Palm applications to the Tapwave platform
code examples
developer program
q & a

European Developer Conference

palmsource

game MARKETWATCH.COM

5.6.2003 2:54:20 PM

New Player Poised to Enter Portable Gaming Market

NORDIC WIRELESS WATCH

Tapwave plays on Palm, reveals Helix handheld

brighthand™

Tapwave Announces Palm Powered Handheld for Serious Gamers

The Mercury News

Handheld aimed at older gamers

By Dean Takahashi
Mercury News

TAPWAVE ES EL NUEVO LICENCIATARIO DEL PALM OS.

Tapwave лицензирует Palm OS и анонсирует игровой гаджет под управлением этой ОСи.

ZDNet UK News
Technology News Now

Start-up Tapwave is developing a portable games console with PDA functionality based on the Palm OS

gamespy.com

Biz Buzz-May 12, 2003
(Tap)Wave Rave

c|net NEWS.COM TECH NEWS FIRST

Gaming start-up gets Palm license

BUSINESS 2.0

Not the Only Game Boy in Town

Look out Nintendo. A startup called TapWave is reinventing the Palm Pilot as a gaming platform.

REUTERS

Ex-Palm Officials Offer Portable Video Game System

Details Emerge on Palm OS Gaming Machine - Tapwave Helix

Tapwave prepares mobile gaming handheld

Device includes personal information management tools

TapWave to introduce the Helix Palm gaming platform

PalmSource announces a new Palm licensee

GEEK.com GameGeek

Tapwave Inc

Mobiileja pelilaitteita kehittävä amerikkalaisyritys, jonka perustivat Palr Inc:ltä lähteneet johtajat.

PalmSource Licenses Palm OS to Tapwave

Newly launched company will create mobile entertainment products

company introduction

Founded in May 2001 by:

- **Peng Lim**, formerly Vice President of Worldwide Product Development for Palm, Inc.
- **Byron Connell**, formerly VP of Consumer Markets Group and acting VP of Marketing for Palm, Inc.

PalmSource licensee since May 2001

Headquartered in Mountain View, CA

company introduction: our purpose

position

Tapwave gives
technology enthusiasts
(primarily ages 18-34)
the most
fun and function
in mobile
entertainment gear.

European Developer Conference
#1 the mobile gaming gap (fun)



Sony PlayStation 2
(Target Age Group:13-34)



Microsoft Xbox
(Target Age Group:16-26)



Nintendo GameCube
(Target Age Group: 12-17)

78.2 Million Units (2000)
101.9 Million Units (2004)

Age 34

Market Gap
No Mobile Gaming Solution for "Game Boy Graduates"

tapwave™

25-50 Million Unit Potential (2004)

Stationary

Mobile

- **Video Game Market: $50B ('01) - $86B ('06)**
- **Gaming hardware in 44% of U.S. households**
- **User profile:**
  - Average Age: 28
  - 47% of male college students play video games
  - 58% of most frequent console players over 18
  - 72% of most frequent PC players over 18





Nintendo Game Boy Advance
(Target Age Group: 6-14)

24.6 Million Units (2000)
50.0 Million Units (2004)
150.0 Million Units since 1989

Age 6

European Developer Conference
#2 the mobile tools gap (function)




- Education Tools



- Graphing Calculators

29.3 Million Units (2001)

Age 8-26



- Education Tools



(Target Age Group: 18-34)

Embedded OS
(Single Purpose)

Age 27-37

Open OS
(Multi Purpose)

Large Target Population:

College Students: **15.2M**

High School Students (9-12): **15.1M**

Elementary Learning (K-8): **38.7M**

Student to PC ratio still only 9 to 1.

Age 38-39

- HandHeld Computers

34.0 Million Units (2004)

Age 40-56

My Games: Console quality
single & multiplayer wireless gaming



introducing zodiac™: mobile entertainment gear!

My Videos: Full Motion



My Music: MP3 Player

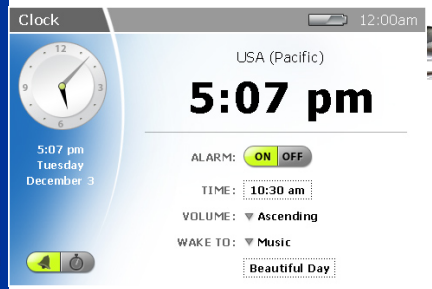




My Album: Photo Viewer



My Time: Music Alarm Clock



My Stuff: Next Gen Launcher



POWERED

European Developer Conference
introducing zodiac …

palmsource

Dual Removable Media & Expansion Slots
SD, Multimedia Card, SDIO (e.g., FM Radio, 802.11b, Digital Camera)

320x480 Display w/ Digitizer
64K colors, 6.8 square inch, side lighting

IR (Infrared)

Left Action Trigger

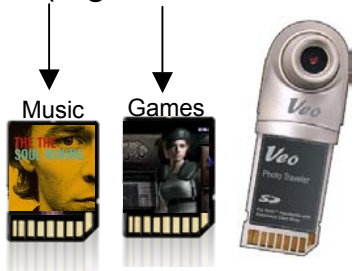
Landscape Design with Pivot to Portrait
Entertainment Ergonomics

Analog Navigation
Plus Selection
(custom skins)

2 Programmable Buttons

Motorola ARM9 Processor & ATI Graphics Acceleration

Headphone Jack







USB Connector & Expansion
PC/Mac Sync & Internet Access

Integrated Bluetooth
Wireless multi-player gaming
& internet access

Right Action Trigger

Rumble/Vibration

Stylus

4 Programmable Buttons

Slim Wearable Design

Stereo Speakers
16-channel, Midi, MP3,
Yamaha Audio Chip

Rechargeable Batteries
Dual Lithium

you've got a vision.
we've got a platform for that vision.

The Zodiac mobile console delivers

- a completely new mobile entertainment experience
- unparalleled game performance not found in any mobile game system

For developers, Zodiac provides a new world of game and application development opportunities

- extend your customer reach and generate incremental revenue
- flexibility for your application development
- Access to a new customer base
- New channels of distribution

next-generation mobile entertainment platform

Combines the best features of the Palm OS with advanced gaming features, including:

- High-resolution, 480 x 320 (half VGA), 16-bit (65,536 colors) color display
- Portrait and landscape display capabilities
- Motorola® i.Mx1™ ARM9 Processor (200 MHz)
- Variable pressure analog controller (joystick), 2 triggers, 4 programmable action buttons, special function button, home button, power button, and Bluetooth button
- Unique ergo-centric design weighing 6.3 ounces
- ATI® Imageon™ W4200 graphics accelerator (with 8MB video memory)
- Yamaha® audio component and stereo speakers, standard 3.5mm stereo headphone plug, with earbud-style headphones included
- High capacity rechargeable lithium batteries -- 1540 mAh
- Dual expansion slots for MMC, SD, and SDIO cards.
- 2 models—32MB and 128MB RAM (See next slide for more information)
- Vibration effects for interactive game play
- Full-featured digital music player
- Bluetooth™ networking for multiplayer wireless gaming

software environment

PalmOS® 5.2T

- Large storage heap
- Graffiti 2

Tapwave Modifications

- Tapwave Native applications
- 10MB dynamic heap + 8MB VRAM + screen buffer
- Improved HotSync – fixes 64K limitation
- New input event queue model
- PINS and status bar support
- Large 480 x 320 double density screen, landscape and portrait modes
- Forward binary compatibility with PalmOS 6.0
- DRM

Bluetooth Motown Café APIs

- Binary compatibility with Palm Bluetooth applications

development tools

Palm OS Simulator

- Supports Landscape and Portrait Screen Modes
- Special Input hardware via GamePad controller
- Large Form layout
- Requires MS Visual C++

Tapwave Native Applications

- Metrowerks CodeWarrior 9.2
- Tapwave SDK

development tools (continued)

Sample Code – “NASCAR” Game Application demonstrates

- Using the Tapwave API to use the full screen size.
- Using the Tapwave Graphics API (TwGfx) for drawing.
- Using the Tapwave Input API (TwInput) to scroll.
- Writing a game as a Tapwave Native Application using CodeWarrior 9.2.
- Interacting with Palm Operating System safely (low battery warnings, alarms, exiting).

API overview

System-level APIs

Graphics APIs

Sound APIs

Gaming Support APIs

Tapwave Native Application APIs

APIs: system-level

New Event Queue Model

- Simultaneous Button Presses
- Analog and/or D-Pad Navigator Input
- Touch sensitive screen input

Screen APIs

- PINS 1.1 from Palm OS for Graffiti, Status Bar, Orientation (Landscape/Portrait)

Virtual Device APIs

- Rumbler

Bluetooth – Motown Café APIs

APIs – graphics and sound

TwGraphics (Low-Level)

- A lightweight fast 2D graphics API
- High performance - a very thin layer around the hardware
- Lightweight - it's easy to use and program
- 2D - provides a common set of 2D rendering operations
- Portable - runs on the device as well as the simulator
- Very fast vram->vram and vram->screen operations
- Alpha blending and transparency support

TwSound (Low-Level)

- Thin layer around sound chip

PalmOS Graphics (Win*) and Sound (Snd*) APIs

Fathammer Libraries

- Device and Platform Independence
- X-Forge Base and Full

APIs: graphics and sound
Fathammer

The X-Forge™ 3D Game Engine is a complete cross-platform mobile game development toolkit that includes:

- Advanced 3D graphics rendering and multi-channel audio
- Collisions and physics
- Character / entity systems and artificial intelligence
- Breathtaking particles, flares and other eye-candy effects
- Multi-player engine for Bluetooth / Wireless LAN gaming
- Special optimizations for different game genres
- Content development and converting tools

Based on established PC and console industry game engine architectures (C++ based, APIs similar to DirectX and OpenGL)

APIs: gaming support

High Score Support w/Conduit & Website

- Simple interface for storing high scores, backup included
- Dual opt-in for web server support
  - Game must request server sharing
  - User must enable server sharing
- Can extend to retrieving high scores from server
- Basis for wider gaming (tournaments, clubs, ladders)

Digital Rights Management

- Secure device serial number in each device
- Based on unchangeable Device, SD Card, or ROM Card IDs
- OS utilities to verify apps that are locked to card, device, or user
- Server utilities to sign applications to card, device, or user
- (Not for data/media rights management)

APIs: tapwave native application (tna)

- Framework for high performance games
- Small 68k stub, the game engine is all native ARM
- 600 native ARM APIs (not PACE wrappers)
- No code segment size limit
- Binary compatibility with future Palm OS releases
- Huge memory allocation limit (8MB in a single chunk)
- Real global variable and relocation support
- Standard C/C++ library support
- See the “HighPerformanceGames.doc” for more info

porting existing PalmOS apps to Tapwave

Intention is 100% compatibility for “well behaved” PalmOS apps

Watch out for:

- Direct screen access
  - 16-bpp double density screen, hardware accelerated
  - base address & origin changes with centering, left-handed mode
  - RowBytes changes with orientation
- Assumptions about input (Palm’s 5-way nav, app/scroll buttons)

Things to Add:

- Enable full screen, landscape preferred, consider centering
- Use analog joystick, triggers, action buttons
- Follow play/pause/exit guidelines

Plan of Attack

- Test your app under the Simulator w/debug ROM
- Talk to us about what you find

code examples

Handling Screen Size Changes: PINS spec

- During **frmLoadEvent/frmOpenEvent**: (initialize your app, must be done per-form)
  - Screen is initially landscape (normally), with graffiti and status open
  - **WinSetContraintSize**(…)
  - **FrmSetDIAPolicy**(formP, frmDIAPolicyCustom)
  - **PINSetInputAreaState**(pinInputAreaShow, -Hide, -User)
  - **StatShow**() or **StatHide**() - standard apps should leave status open
  - **SysSetOrientation**(sysOrientationLandscape, -Portrait, -User)
  - **WinGetBounds**(**WinGetDisplayWindow**(), &bounds)
  - Perform all of the above BEFORE calling FrmDrawForm to reduce flicker.

code examples

Handling Screen Size Changes: PINS spec (continued)

- During **winDisplayChangedEvent**: (graffiti, status, orientation change)
  - Compute deltaX and deltaY: compare form bounds with display bounds
  - Update your form size: **WinGetBounds**(**WinGetDisplayWindow**(), &bounds), **WinSetBounds**(**WinGetActiveWindow**(), &bounds);
  - Change form object positions: **FrmGetObjectBounds, FrmSetObjectBounds**
- During **winExitEvent**:
  - Is finally sent reliably!
  - Games should pause, and release TwInput
- During **winEnterEvent**:
  - If necessary, set up graffiti, status, and orientation
    - OS will restore graffiti, status, and orientation if you return **handled=false** or call **FrmHandleEvent**
- See article 507 in the PalmSource Knowledge Base for more information (kb.palmsource.com).

code examples

User Input (Analog Joystick, Game Buttons)

- **TwInputOpen**(&queueP, “twinput”, “r”), **TwInputClose**(queueP)
- **TwInputSetFormat**(queueP, &format, formatSize * sizeof(Int32))
  - Format is just an array of **Int32**s, each is a requested data type
  - Formats specify what data you want to read, subset of:
    - Pen state: X, Y, Up or Down. (Only over app area)
    - Joystick state: cartesian X, Y or polar R, Theta
    - Joystick press (“Select” button): Up or Down
    - Virtual 4-way or 8-way rocker: Up/Down/Left/Right + Center
    - Action Buttons: A, B, C, D; Up or Down
    - Triggers: Left, Right; Up or Down
    - Function button: Up or Down
- **TwInputPoll**(queueP, &myEvent, sizeof(MyEventType)
  - **MyEventType** is a struct of **Int32**s that matches the format
  - Simultaneously gives current state of all controls
- Check out the “Navigation” sample for more information

code examples

User Input (Fast Queues)

- **TwInputActivate**(queueP) to start queueing (not needed if you poll)
- **TwInputDeactivate**(queueP) to stop queueing
- **TwInputRead**(queueP, *myEvent, sizeof(MyEventType))
  - Reads data from the queue
- **err = TwInputPeek**(queueP, *myEvent, sizeof(MyEventType))
  - **err = EAGAIN** if there are no events, myEvent is optional
  - does not dequeue event
- **TwInputSetPeriod**(queueP, milliseconds) to control poll frequency
  - Not needed if you only want data when a control changes state
- **TwInputSetCapacity**(queueP, capacity) to set queue size
  - Controls how many events to queue
  - Any transition is a whole new event
  - Default is 0 (only current state), max is 16.

code examples

TwGraphics Example

- General APIs
  - **TwGfxOpen**() and **TwGfxClose**() to get access to the graphics library
  - **TwGfxWaitForVBlank**() to wait for VBLANK to begin
- Surface Manupulation APIs
  - **TwGfxAllocSurface**(gfxType, &result, gfxSurfaceDescription) to allocate a new surface, for use by rendering calls. **TwGfxFreeSurface**() to free.
  - **TwGfxGetSurfaceInfo**() to get info about a surface
  - **TwGfxGetPalmSurface**() to get handle to surface representing the display
  - **TwGfxSetClip**(), **TwGfxGetClip**() to set and get clipping rectangle on surface
  - **TwGfxLockSurface**() and **TwGfxUnlockSurface**() to lock surface in memory
  - **TwGfxWriteSurface**() writes buffer to display
  - **TwGfxReadSurface**() makes a copy of the surface display memory
  - Etc…

code examples

TwGraphics Example (cont)

- Rendering Calls
  - **TwGfxBitBlt** (destSurface, destPoint, srcSurface, srcRect)
  - **TwGfxTransformBlt** (destSurface, destPoint, srcSurface, srcRect, rotationFlags, mirrorFlags) to mirror and rotate in 90-degree increments
  - **TwGfxDrawPoints**(destSurface, &points, numPoints, color) to draw points
  - **TwGfxDrawColorPoints**(destSurface, &points, numPoints, &colors)
  - **TwGfxDrawLines**(destSurface, &points, numPoints, color)
  - **TwGfxDrawLineSegments**(destSurface, &points, numPoints, color)
  - **TwGfxDrawRect**(destSurface, rect, color)
  - **TwGfxFillRect**(destSurface, rect, color)
  - **TwGfxDrawSpans**(destSurface, &spans, numSpans, color)
  - **TwGfxDrawBitmap**(destSurface, destPoint, bitmap, transparentBG)
  - **TwGfxStretchBlt**(destSurface, destRect, srcSurface, srcRect)
  - TwGfxBlendBlt(destSurface, destPoint, srcSurface, srcRect, alpha)
  - Etc…

code examples

Fathammer Game Example

See the X-Forge SDK for documentation and examples.

code examples

Vibration example

- **TwDeviceOpen**(&deviceHandle, “rumbler0”, “rw”)
- **TwDeviceClose**(deviceHandle)
- **TwDeviceControl**(deviceHandle, cmd, &buffer, length)
  - Cmd = **strt**, **stop**, **play**, **rept**, **fast**, **medi**, **slow**
  - Buffer = array of UInt8 pairs: speed and duration, for **play** and **rept**
  - Length = number of array elements in bytes
- Can turn vibrate on “forever” using **strt/fast**, **medi**, **slow**
- Turn vibrate off no matter what using **stop**
- Can play an effect from a library using **play** or **rept**
  - Speed is 0-255, 0 = off
  - Duration is in 1/100ths of a second
  - Example fast, slow, fast:
    [‘\255’, ’\10’, ‘\0’, ‘\10’, ‘\127’, ’\10’, ‘\0’, ‘\10’, ‘\255’, ’\10’, ‘\0’, ‘\10’]

code examples

High Score example

- **TwHighScoreRegister**(myCreatorID, 0 /* score type */,
  10 /* local scores */, 0 /* server scores */,
  false /* write scores to server */, “MyCoolGame")
  - Only needs to be called once, but no harm in calling it every launch
- **TwHighScoreUnregister**(myCreatorID, 0)
  - Deletes all high score data, only called during “delete me” cleanup
- **TwHighScoreReport**(myCreatorID, 0, &highScoreData)
  - Data contains UInt32 score, DateTime, and a game-specific string
- **TwHighScoreGetSummary**(myCreatorID, 0)
  - Returns number of scores, highest/lowest, server & local, sync time
- **TwHighScoreGetDetails**(myCreatorID, 0, Local/Server, index)
  - Returns details about an individual score entry

code examples

Digital Rights Management example

- **TwSecGetFunctions**(version, *codeSignature, **codeStart, *codeSize)
  - Returns a pointer to the DRM functions. Make sure the pointer is from ROM and that codeSignature is correct.
- **TwSecGetPublicKey**(*publicKey, keyNumber)
  - Returns the device's public keys.
- **TwSecGetHardwareId**(*hardwareId, hardwareInfoType, slotNum)
  - Returns the id of a given type of hardware.
- **TwSecVerifyDatabase**(cardNo, dbId, *failureReason, failureAction)
  - Verifies security information on a database.
- **TwSecVerifyCurrentApp**(*failureReason, failureAction)
  - Verifies security information of an application.
- **TwSecVerifyMemory**(*mem, memSize, *key, *sig, *failureReason, failureAction)
  - Verifies a block of memory.
- **TwSecFailureReset**(*filename, reason, *message)
  - Reset device and display reason after reset.

code examples

Tapwave Native Application Example

- **TwAppStart**(&eventHandler)
  - Sets up TNA world, creates base form, sets event handler
- **TwAppStop**()
  - Tears down TNA world
- **TwAppRun**(timeout)
  - Runs event loop, calls **EvtGetEvent**, **FrmDispatchEvent**, **SysHandleEvent**, **MenuHandleEvent**, **FrmDispatchEvent**

Get Started -- Join the Tapwave developer program

Benefits of Membership

- Easy and free to get started
- Access to X-Forge 3D game engine at reduced fees
- Wide range of developer support
- Tapwave developer's kit includes all the documentation, header files and sample code you need to develop for Tapwave
- Opportunity to sell and market your "Tuned for Zodiac" or "Certified for Zodiac" application in our online store
- Special promotional opportunities
- Discounts on development devices

Developer Program

Product Testing & Certification







full screen



bluetooth



analog controller



tournament



vibration



multiplayer

Developer Communications

How do I get started:

Go to www.tapwave.com/developers to complete the application and SDK agreement

Contact Information:

Joyce Morrell, Director, Developer Programs
Developer program and business opportunities: joyce@tapwave.com

Ryan Robertson, Developer Technical Support
developer@tapwave.com

q & a



go. do. play.